4COSC002W Mathematics for Computing

Lecture 1

Module Introduction. Number Formats. Types of Numbers. Number Theory Basics. Modular Arithmetic. Sequences.

UNIVERSITY OF WESTMINSTER

WELCOME
TO THE MATHS FOR COMPUTING MODULE!

Module Leader:
Dr Natalia Yerashenia
Lecturer in Data Science & Analytics

n.yerashenia3@westminster.ac.uk
(please use your university email to contact me and avoid Blackboard messages)

Office: 7.115, Cavendish Campus

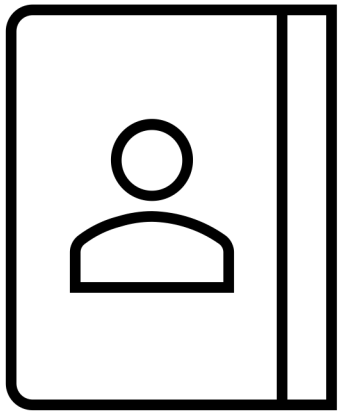
[UoW Profile Link]

Office Hours:

- Tue 12:00–13:30
- Thu 16:15–17:15

Book an office hours appointment with me via [Doodle]

Module Team 2024-2025



Kamran Javidi K.Javidi@westminster.ac.uk (module co-leader)

Dr Anastasia Angelopoulou A.Angelopoulou@westminster.ac.uk
Dr Andrea Martina A.Martina@westminster.ac.uk
Dr Nalaka Dissanayake N.Dissanayake@westminster.ac.uk
Dr Olivier Moullard O.Moullard@westminster.ac.uk
Dr Vitaly Fain V.Fain@westminster.ac.uk

Module Organisation

- Module Duration: One Semester, 12 Weeks
- Weekly Online Lectures (2hrs) & In-Class Tutorials (2hrs)
- All the module-related materials will be available on Blackboard.
- Make sure you check your university emails regularly to follow the important announcements.
- Attendance is essential! Don't forget to swipe into all your timetable classes using your Student ID cards. More info [here].

!MODULE ASSESSMENT!

Two Formal Assessments: ICT & Exam. No Coursework.

- **In-Class Test (ICT)** – Week 7. In-Class Test. Duration: 90 minutes. Closed book. Multiple choice questions. Mock Test available in Week 5. Optional Online Revision Session during Week 6 (Engagement Week).
- **Exam** – Written Exam in January. [University Exam Timetable]. Organised by the School Registry. Parts A & B. Part A - Short Answers. Part B – Full Detailed Answers. Duration: 120 minutes. Mock Exam Paper available in Week 11. Revision during Lecture 12.

Calculations: Overall Module Mark = 0.4*ICT+0.6*Exam. The Overall Module Pass mark is 40. Components Pass mark – 30.

General Information on the Assessment Regulations can be found in the university's [Handbook of Academic Regulations]

Module Syllabus

- Number Theory Basics
- Sequences. Set Theory. Intervals
- Relations and Functions
- Logics Basics
- Fundamental Data Structures
- Graph Theory
- Matrices
- Probability Theory
- Elementary Statistics
- Math Related Applications

Sources

Blackboard (BB) e-learning platform

Lecture Notes, Lecture Video Recordings, Tutorial Tasks & Answers, Module Announcements, Discussion Board etc.

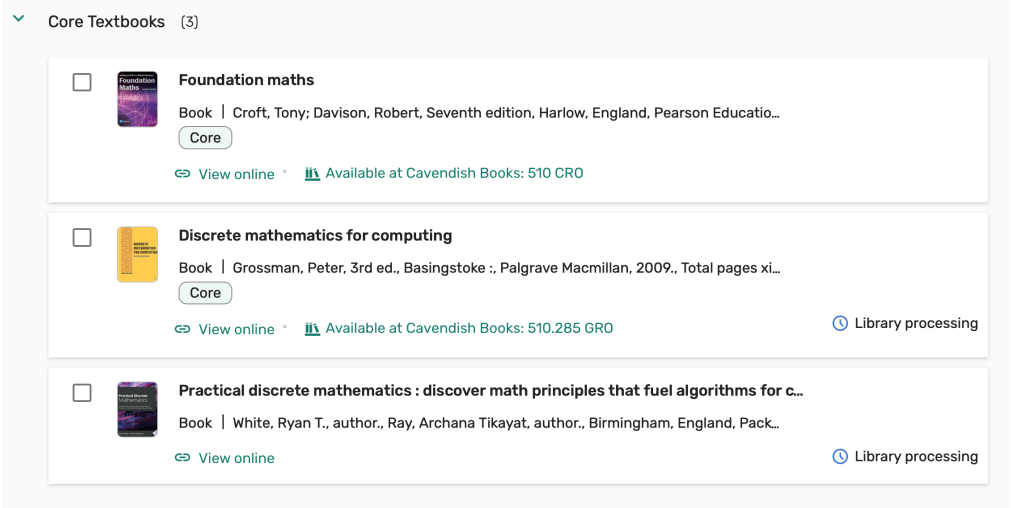
Core Textbook: Croft & Davison *Foundation Maths*

Further Reading: see [Reading List] on BB

[YouTube Videos] – the same link is on BB

Useful Links: Articles, Websites, Courses, and Applications

check for the regular updates on BB



Lectures

24 teaching & learning hours

Purpose of the Lectures: To ensure all students have the necessary mathematical foundation for advanced Computer Science modules

Intensive Format: Each week will cover a new topic

Content Focus: Theoretical essentials with practical examples

Delivery Mode: Online Pre-recorded (released on Tuesday afternoon) + Online Q&A Session (each Friday morning)

Lecture Guidelines:

- Attendance is important!
- Make notes during the lecture and Q&A session
- Prepare your questions for the Q&A session during tutorials and while watching the lecture recordings

Seminars

24 teaching & learning hours

Foundation Mathematics Revision during Week 1

Content Focus: Practical tasks based on the previous week's lecture material

Detailed **answers** will be published on Blackboard at the end of each week

To prepare for the seminars, check:

- Lecture notes
- Textbook chapters
- Additional theoretical material on the related topic (provided on BB)

Seminar Structure

- A quick recap of the theory material related to the seminar tasks
- Attempting tasks on your own or as part of a group
- Discussion of the tasks with your tutor; Q&A
- Unfinished tasks will be assigned as homework

Each seminar focuses on a specific topic. Clarifying doubts about the topic with your tutor before the semester ends is essential.

IMPORTANT: Seminar tasks are closely related to formal assessment tasks.

Handwriting notes (using pen & paper or a tablet) is a MUST as good practice for the upcoming exam.

Independent Study

152 teaching & learning hours

According to the module proforma, you must dedicate yourself to learning this module outside the scheduled classes.

MAKE YOUR OWN NOTES!

Taking notes will greatly simplify the revision process

Form **Self-Study Groups**

Collaborating with classmates can make learning less stressful and more effective

Use **time management tools**

Tools like Notion, Trello, or similar can help organise your study schedule efficiently

ChatGPT is an excellent helper for clarifying topics you don't understand. But remember, the formal assessments are closed-book. No source of information (except your knowledge) is allowed during the assessments

Use **additional sources of information**

Textbooks (check the module reading list)

YouTube, and online free courses.

For example, [MIT Mathematics for Computer Science] video course. Also, UoW provides free access to [LinkedIn Learning].

Let's get closer to mathematics...

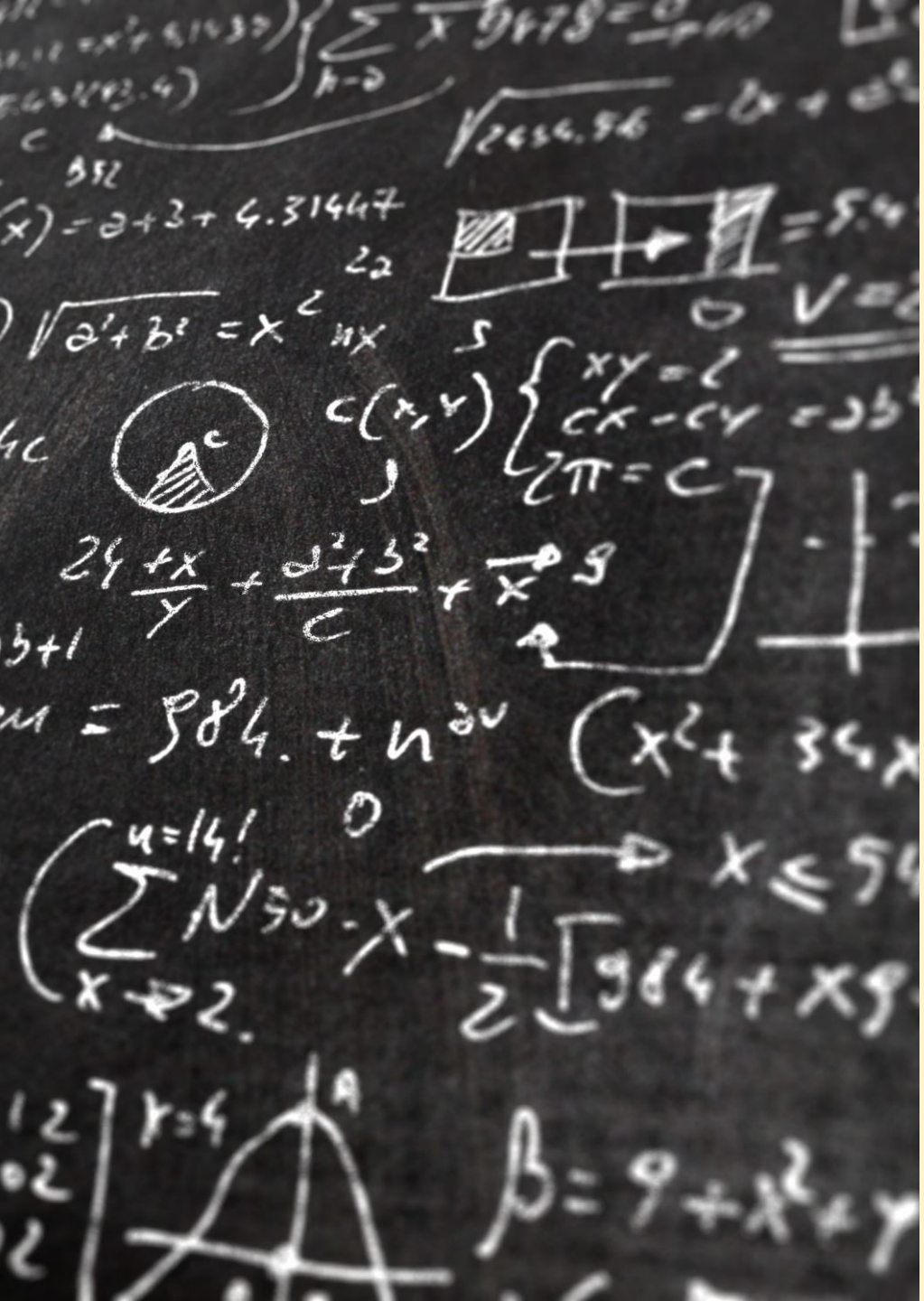
Mathematics as the Fundamental Concept of Computer Science

Q&A SESSION ACTIVITY:
Word Cloud

Question: What Mathematics topics have you learned before?

Share your background experience

PollEv.com/nataliayerashenia404



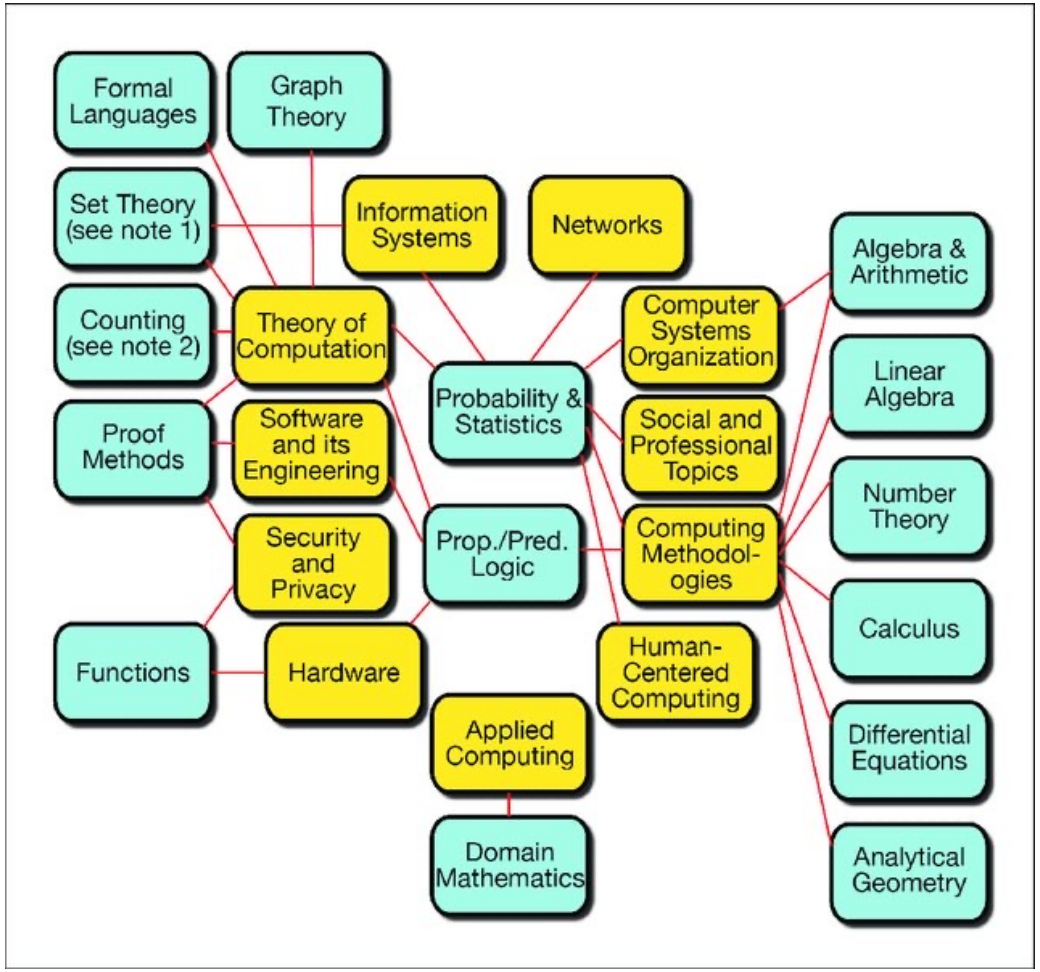
ACTIVITY:
Home Revision Task

Key Topics to Review

- **Arithmetic Skills**: Decimals, fractions, percentages, absolute values, order of operations.
- **Algebra Foundations**: Algebraic notation, terms and factors, solving equations, rearranging formulae, inequalities.
- **Algebraic Operations**: Simplifying expressions, expanding brackets, factorising expressions.
- **Ratios and Proportions**: Applications with percentages and indices.
- **Exponents and Logarithms**: Laws of powers and logs, solving exponential and logarithmic equations.
- **Units**: Standard units and compound units in calculations.
- **Equations**: Solving linear equations, equations with fractions, quadratic equations (factorisation, completing the square, quadratic formula).

IMPORTANT: Skim over **CHAPTERS 1-10 of the Croft & Davison Foundation Maths textbook** to make sure that you know *the maths basics* required for the module

Mathematics associated with computer science areas



Source: Baldwin, D., Walker, H.M. and Henderson, P.B., 2013. The roles of mathematics in computer science. *Acm Inroads*, *4*(4), pp.74-80.

Why do Maths for Computing is essential for programming?

- **LOGICAL REASONING:** Maths sharpens the ability to think logically and solve problems, essential for writing efficient algorithms and code.
- **ALGORITHMIC FOUNDATION:** Many algorithms are based on mathematical principles and formulas.
- **DATA STRUCTURES:** Concepts like sets, graphs, and trees, foundational in maths, are vital in creating efficient data structures.
- **MACHINE LEARNING/AI:** Requires understanding of statistics, probability, linear algebra, and calculus.
- **COMPUTER GRAPHICS:** Geometry and trigonometry are fundamental for rendering images and animations.
- **CRYPTOGRAPHY & SECURITY:** Number theory and modular arithmetic form the basis of cryptographic algorithms.
- **EFFICIENCY AND OPTIMSATION:** Maths helps analyse and improve code efficiency through big O notation and other methods.

Discrete Mathematics

What is Discrete Mathematics?

- A branch of mathematics that deals with distinct, separate, or non-continuous structures.

Main Topics in Discrete Math:

- **Logic**: Propositions, truth tables, predicates, and quantifiers.
- **Set Theory**: Basics of sets, operations, and relations.
- **Combinatorics**: Counting, permutations, and combinations.
- **Graph Theory**: Vertices, edges, trees, and traversal.
- **Number Theory**: Prime numbers, divisibility, and modular arithmetic.
- **Algorithms**: Analysis, sorting, and searching.

Discrete Mathematics provides the essential theoretical foundation for many areas of computer science and informs our understanding of the digital world.

Number Formats & Types

Numbers in Computer Science

Number Formats: Decimal Number System

The **decimal number system** (base-10) is the standard system for representing numbers, using the digits 0 through 9. Each position in a decimal number has a **place value** based on powers of 10, making it intuitive for everyday use.

Base-10: The decimal system uses **10 digits** (0–9).

Place Value: Each digit's position determines its value, increasing by powers of 10 as you move left.

Example:

The number **3471** represents:
$3 \times 10^3 = 3000$
$4 \times 10^2 = 400$
$7 \times 10^1 = 70$
$1 \times 10^0 = 1$

Total: $3000+400+70+1=$**3471**

Number Formats: Binary Number System

The **binary number system** is a **base-2 numeral system** that uses only two digits: 0 and 1. Regardless of size, every number can be represented in binary form, making it the fundamental numbering system for computers and digital systems.

Examples:

- **14 in binary**: 1110_2
- **245 in binary**: 11110101_2
- **34567 in binary**: 1000011100001111_2

All data in a computer is stored in binary, whether it's text, images, or numbers.

Why Binary? Computers use binary because they operate using electrical signals with two states: ON (1) and OFF (0). This makes binary the most efficient way to represent and process data electronically.

Memory consists of individual bits (binary digits), each of which is either a 0 or 1. These bits are grouped into bytes (8 bits), and sequences of bytes are used to store more complex data, such as numbers, characters, or larger structures like arrays and files.

Conversion Between Number Systems

Decimal to Binary Conversion: Repeatedly divide the decimal number by 2, keeping track of the remainders. The binary number is the sequence of remainders read from bottom to top.

Example: Convert **19** to binary.

Divide by 2 and keep track of the remainders:

1. $19 \div 2 = 9$ remainder **1**
2. $9 \div 2 = 4$ remainder **1**
3. $4 \div 2 = 2$ remainder **0**
4. $2 \div 2 = 1$ remainder **0**
5. $1 \div 2 = 0$ remainder **1**

Result: Reading the remainders from bottom to top. **19 in decimal** is **10011 in binary**.

Binary to Decimal Conversion: Multiply each binary digit by 2^n, where n is the position of the digit, starting from 0 from the right.

Example: Convert **10011** back to decimal.

Start from the rightmost bit and multiply each bit by powers of 2, based on its position:

1. $1 \times 2^4 = 16$
2. $0 \times 2^3 = 0$
3. $0 \times 2^2 = 0$
4. $1 \times 2^1 = 2$
5. $1 \times 2^0 = 1$

Result: Add them together.$16 + 0 + 0 + 2 + 1 =$ **19**

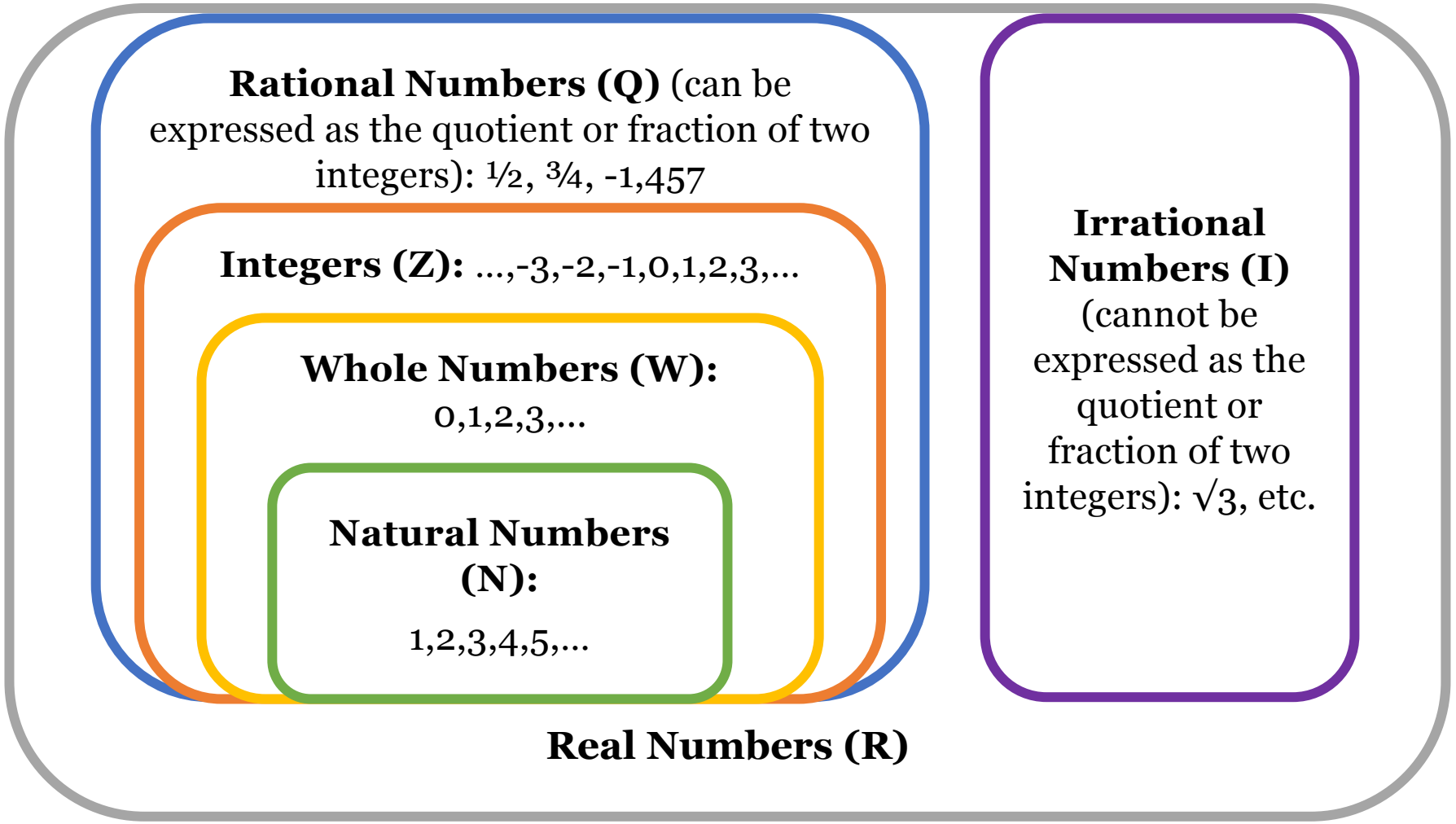
Binary Storage in Memory:

```
00000000 00000000 00000000 00010011 (on a 32-bit system)
```

Decimal to Binary Table (1–10)

Decimal	Binary
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010

Number Types



Number Sets in Mathematics and Their Programming Equivalents

Mathematical Set	Example Values	Typical Programming Data Type	Notes
ℕ (Natural numbers)	0, 1, 2, 3…	`int` (integer)	Computers don't distinguish between "positive only" and "all integers."
ℤ (Integers)	−2, −1, 0, 1, 2	`int` (integer)	Usually stored in fixed-size formats (e.g. 32-bit, 64-bit).
ℚ (Rational numbers)	½, −7/3	`fractions.Fraction` (Python), custom class, or two `int`s (numerator/denominator)	Not built-in in most languages; usually represented as a pair of integers.
ℝ (Real numbers)	3.14, −0.01, √2	`float` or `double` (floating-point)	Computers can only approximate real numbers due to finite memory.
ℂ (Complex numbers)	2 + 3i	`complex` (Python), `Complex` class (Java, C#)	Not always built-in, but available in some languages.

Introduction to Number Theory

Number Theory Basics

Introduction to the Number Theory

NUMBER THEORY is a branch of pure mathematics devoted to the study of integers and, more generally, to objects built out of them.

Integers (ℤ) are the set of whole numbers and their negatives, including zero. In other words, they extend *whole numbers* by adding the *negative* counterparts. The symbol ℤ comes from the German word "Zahlen", meaning "numbers."

Formally: $Z=\{\ldots,-3,-2,-1,0,1,2,3,\ldots\}$

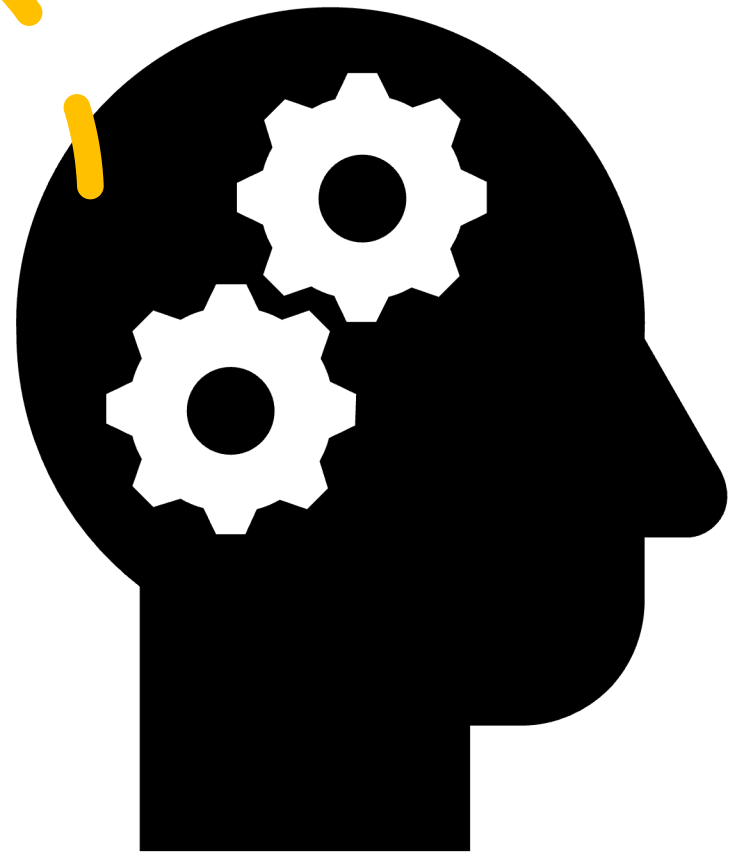
Prime and Composite Numbers

- **Prime Numbers**: Natural numbers greater than 1 that have no positive divisors other than 1 and itself.
- **Composite Numbers**: Natural numbers greater than 1 that are not prime.

Example: 5 is prime because it can only be divided by 1 and 5, but 4 is composite because it can be divided by 1, 2, and 4.

Why are Integers popular in coding?

- **Efficiency**: Integers are stored and processed more quickly by the CPU compared to floating-point numbers.
- **Exact Values**: Unlike floating-point numbers, integers represent exact values without rounding errors.
- **Memory Usage**: Integers require less memory compared to more complex data types like floats or doubles.
- **Looping and Indexing**: Commonly used to control loops and index arrays due to their precision and simplicity.
- **Bitwise Operations**: Integers are ideal for bit-level operations (AND, OR, XOR) and shifting, commonly used in low-level programming.
- **Compatibility**: Widely supported by all programming languages and hardware architectures.



Fundamental Theorem of Arithmetic

Statement: Every integer greater than 1 is either a prime number itself or can be factorised uniquely into prime numbers up to the order of the factors.

Implication: Prime numbers are the "building blocks" of all natural numbers.

The factorisation is decomposing a number into a product of other numbers.

In the context of the Fundamental Theorem of Arithmetic, factorisation refers specifically to representing an integer as a product of prime numbers.

Example of Factorisation:

Consider the number 84. We can factorise it as follows:

$84=2\times42=2\times(2\times21)=2\times2\times(3\times7)=2^2\times3\times7$

Division Theorem

For two integers a and b where $b \neq 0$, there always exists unique integers q and r such that $\boldsymbol{a = qb + r}$ and $0 \leq r < |b|$.

Example:

$a = 17$, $b=3$, we can find $q = 5$ and $r = 2$ so that $17 = 3^*5+2$

a is called the **dividend**
b is called the **divisor**
q is called the **quotient**
r is called the **remainder**

$\frac{A}{B}$ = Q rem R

If $r = 0$, then we say b **divides** a or a is **divisible** by b.

Modular Arithmetic Basics

Modular arithmetic is a system of arithmetic for integers, where numbers "wrap around" after reaching a certain value, known as the modulus.

In modular arithmetic, we focus on the remainder r when dividing a by b.

Modular Arithmetic Notation: **A mod B = R** *(vs. Division Theorem:* $\frac{A}{B} = Q\ rem\ R$*)*

Example:

(1) $14 \bmod 5 = 4$ (since $14 = 2 \times 5 + 4$ or $\frac{14}{5} = 2\ rem\ 4$)

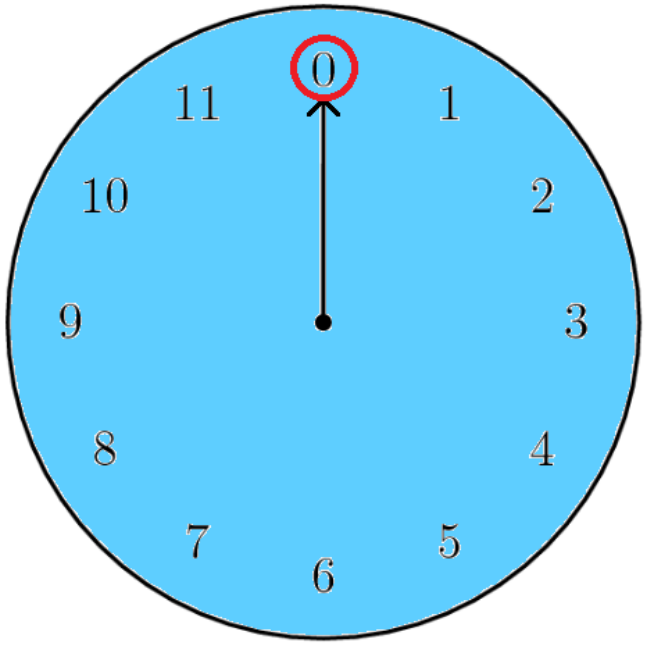
We also say $14 \equiv 4 \pmod 5$ That means 14 is congruent to 4 modulo 5. In fact, any number that leaves remainder 4 when divided by 5 is congruent to 14 modulo 5. So, the full set of numbers congruent to 14 is: {…,−6,−1,4,9,14,19,24,… } In general: $14 \equiv 4 \equiv 9 \equiv 19 \equiv 24 \equiv \ldots \pmod 5$

(2) $23 \bmod 7 = 2$ (since $23 = 3 \times 7 + 2$ or $\frac{23}{7} = 3\ rem\ 2$)

The full set: {…,−12,−5,2,9,16,23,30,37,…}, $23 \equiv 2 \equiv 9 \equiv 16 \equiv 30 \equiv 37 \equiv \ldots \pmod 7$

Key Idea: Numbers are treated as equivalent if they leave the same remainder when divided by b.

Modular Arithmetic Examples



12hrs Clock

8 hrs +10 hrs =What time is it?

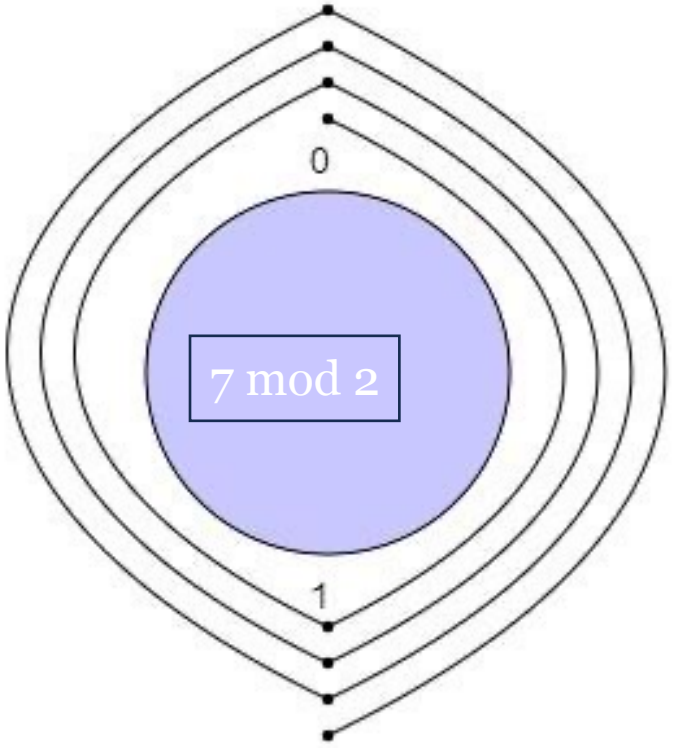
8 hrs +22 hrs=???
22 hrs= 10hrs+12hrs

U.K. schools getting rid of analogue clocks because teens “cannot tell time.”

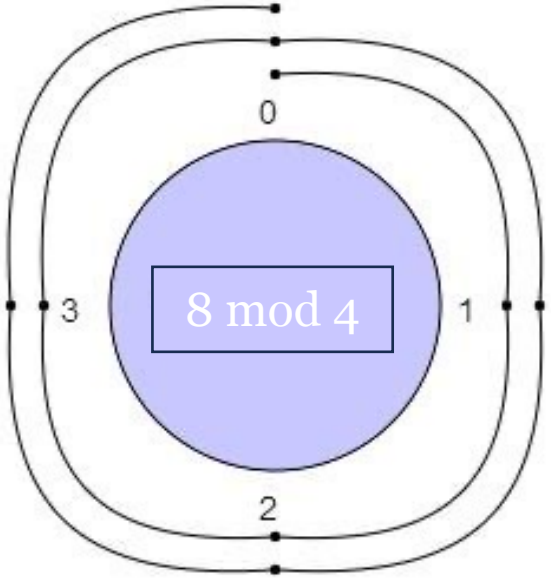
Source: CBS News, May 8, 2018

Modular Arithmetic Examples

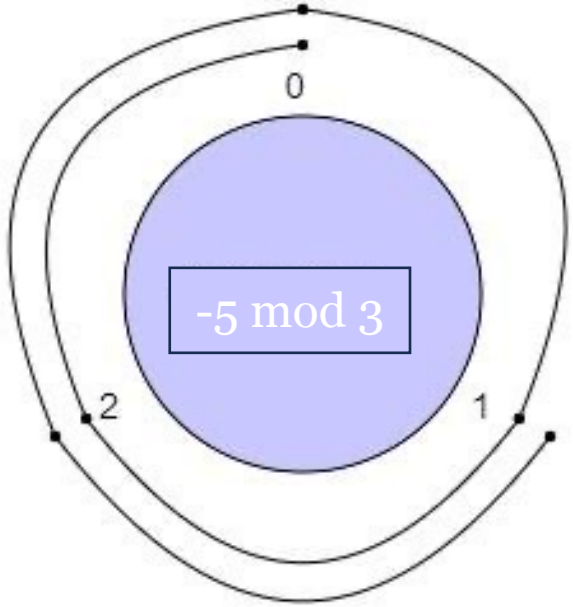
7 mod 2=1

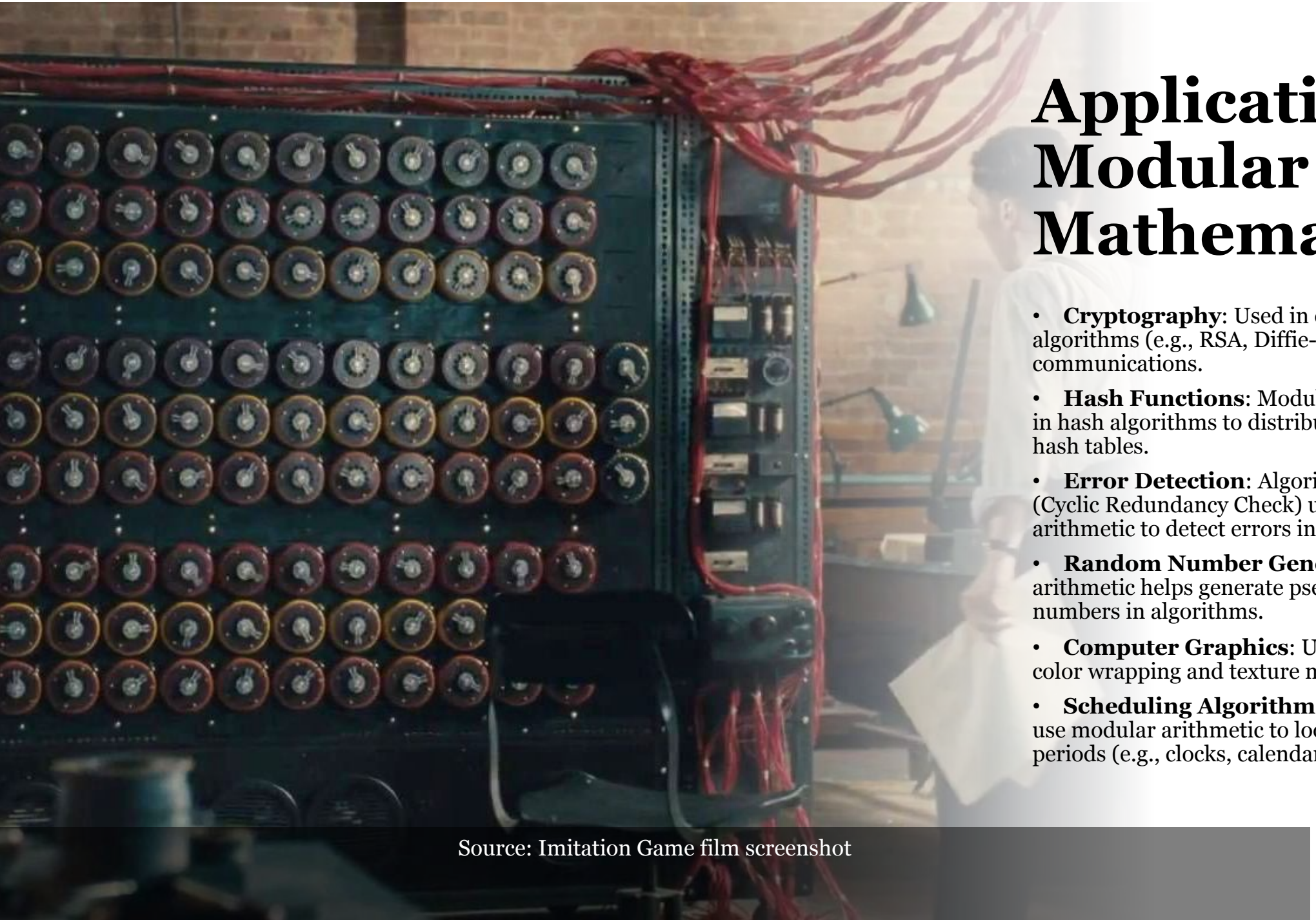


8 mod 4 = 0



-5 mod 3 =1





Source: Imitation Game film screenshot

Applications of Modular Mathematics

- **Cryptography**: Used in encryption algorithms (e.g., RSA, Diffie-Hellman) to secure communications.
- **Hash Functions**: Modular arithmetic is used in hash algorithms to distribute data evenly in hash tables.
- **Error Detection**: Algorithms like CRC (Cyclic Redundancy Check) use modular arithmetic to detect errors in data transmission.
- **Random Number Generation**: Modular arithmetic helps generate pseudo-random numbers in algorithms.
- **Computer Graphics**: Used in algorithms for color wrapping and texture mapping.
- **Scheduling Algorithms**: Time-based tasks use modular arithmetic to loop within fixed periods (e.g., clocks, calendars).

HOMEWORK ACTIVITY: Codebreakers

Task: Decrypt a Simple Caesar Cipher Message Manually

The Caesar Cipher shifts each letter in the alphabet by a fixed number (key). The *encryption* equation is: $C \equiv P + k \bmod 26$,

where C is the ciphertext letter, P is the plaintext letter, k is the shift value (key).

26 represents the number of letters in the English alphabet.

Given: You are given an encrypted message: **"UPQY"**—the shift value (key) k=2.

Decrypt the given message manually on paper.

Hints: Write down the corresponding numeric values for each letter in the encrypted message (U=20, P=15, Y=24, Q=16). Use the *decryption* equation: $P \equiv C - k \bmod 26$

Binary Number System and Modular Arithmetic

The **binary number system** (base-2) and **modular arithmetic** are closely related.

Modular arithmetic operates on numbers with a fixed range, much like how binary numbers wrap around when adding or subtracting (mod2).

Sequences

A sequence is an ordered list of elements (numbers, objects, etc.), arranged according to a certain rule or pattern.

Each element in a sequence is called **a term**, and the position of a term in a sequence is called its **index**.

Example

(2,4,6,8,10,12) – sequence of even numbers

(1,3,5,7,9,11) – sequence of odd numbers

Sequences can be *finite* or *infinite*.

Arithmetic Sequence

An arithmetic sequence (progression) is a sequence of numbers in which the difference between consecutive terms is constant. This difference is called the "common difference," denoted by d. The general form of an arithmetic sequence is

a, a+d, a+2d, a+3d, ..., here, a is the first term of the sequence.

The n^{th} term of an arithmetic sequence can be calculated using the formula:

$$a_n = a + (n-1)d$$

Example:

In the sequence **2, 5, 8, 11, ...,** the common difference d is **3**.

Geometric Sequence

- **A geometric sequence** (progression) is a sequence of numbers where each term after the first is found by multiplying the previous term by a fixed, non-zero number called the "common ratio," denoted by r. The general form of a geometric sequence is:

$a, ar, ar^2, ar^3, \ldots$, here, a is the first term of the sequence.

The n^{th} term of a geometric sequence can be calculated using the formula:

$$a_n = a \cdot r^{(n-1)}$$

Example:

In the sequence **3, 6, 12, 24, …** the common ratio r is **2**.

Fibonacci Sequence. Divine Ratio

The Fibonacci sequence is a series of numbers in which each is the sum of the two preceding ones, usually starting with 0 and 1. In mathematical terms, the sequence F(n) of Fibonacci numbers is defined by the recurrence relation:

$$F(n)=F(n-1)+F(n-2),$$

with initial conditions: F(0)=0, F(1)=1

0,1,1,2,3,5,8,13,21,34…

Golden (Devine) Ratio: The ratio of consecutive Fibonacci numbers converges to the Golden Ratio. $\varphi \approx 1.61803398874895$. This ratio appears in various aspects of art, architecture, and nature.

$$\lim_{n\to\infty}\left(\frac{F_{n+1}}{F_n}\right) = \varphi$$